

Servo ansteuern – Kurzreferenz & Muster-Sketch

Modellbauservo SG90 als Stellglied und als analoge Anzeigenadel — 18. September 2026

1 – Was ein Servo ist

Ein Modellbauservo ist ein kompletter Regelkreis in einem Gehäuse: Gleichstrommotor, Getriebe, Potentiometer als Winkelsensor und eine kleine Elektronik. Man gibt ihm keinen Strom und keine Drehzahl vor, sondern einen **Sollwinkel**. Die Elektronik im Servo vergleicht ihn ständig mit dem Istwinkel und fährt nach. Der Stellbereich beträgt bei einem SG90 etwa 0° bis 180°.

Der Sollwinkel wird als **Pulsweite** übertragen: Alle 20 ms (50 Hz) kommt ein Impuls.

Pulsdauer	Sollwinkel	
ca. 1,0 ms	0°	linker Anschlag
ca. 1,5 ms	90°	Mittelstellung
ca. 2,0 ms	180°	rechter Anschlag

Diese Impulse muss man nicht selbst programmieren – die Bibliothek **Servo.h** erzeugt sie im Hintergrund über einen Timer. `write(winkel)` setzt lediglich die Pulsdauer; das Programm läuft sofort weiter und wartet nicht, bis der Servo angekommen ist.

2 – Verdrahtung

Ader am Servo	Farbe (typisch)	Anschluss
Masse	braun oder schwarz	GND
Versorgung	rot	5 V
Signal	orange oder gelb	beliebiger Digitalpin, hier D9

Stromversorgung

Ein einzelner SG90 darf im Unterricht am 5 V-Pin des Arduino hängen. Sobald er unter Last ruckt, die Nadel zittert oder der Arduino sich unvermittelt neu startet, ist der Anlaufstrom zu groß: dann ein eigenes 5 V-Netzteil oder Batteriepack verwenden. In diesem Fall müssen die **Massen von Arduino und Servo-Netzteil verbunden** werden, sonst hat das Signal keinen Bezug.

3 – Das kleinste lauffähige Programm

```
#include <Servo.h>

Servo meinServo;

void setup() {
  meinServo.attach(9);      // Signalpin festlegen
}

void loop() {
  meinServo.write(0);        // Sollwinkel 0 Grad
  delay(1000);               // Zeit zum Hinfahren lassen
  meinServo.write(180);      // Sollwinkel 180 Grad
  delay(1000);
}
```

Die `delay()`-Zeilen sind hier keine Kosmetik: `write()` kehrt sofort zurück, der Servo braucht aber etwa 0,1s bis 0,2s für 60°. Ohne Wartezeit bekäme er den nächsten Sollwert, bevor er den alten erreicht hat – die Nadel zuckt nur.

4 – Poti steuert Servo (Grundmuster aus AB 5)

```
#include <Servo.h>
Servo meinServo;

void loop() {
  int wert = analogRead(A0);           // 0 ... 1023
  int winkel = map(wert, 0, 1023, 0, 180); // auf Winkelbereich abbilden
  meinServo.write(winkel);
  delay(20);                           // ein Rahmentakt
}
```

5 – Muster-Sketch: Servo als Zeigerinstrument

Für den Füllstandsgeber (AB 7) wird nicht der volle ADC-Bereich abgebildet, sondern nur der tatsächlich genutzte Bereich zwischen „leer“ und „voll“. Das ist genau die Kalibrierung, die auch der LED-Balken braucht – nur endet die Kette hier in einem Winkel statt in einer Leuchtdiode.

```
#include <Servo.h>

Servo zeiger;

const byte POTI_PIN = A0;
const byte SERVO_PIN = 9;

const int ADC_LEER = 620;           // eigene Kalibrierwerte eintragen
const int ADC_VOLL = 993;

const int WINKEL_MIN = 5;           // Reserve vor den Endanschlaegen
const int WINKEL_MAX = 175;

const byte ANZAHL_MESSUNGEN = 10;   // Mittelwert gegen ADC-Rauschen
const int TOTBAND = 2;               // Grad; darunter Nadel nicht bewegen

int letzterWinkel = -100;

void setup() {
  zeiger.attach(SERVO_PIN);
  Serial.begin(9600);
  zeiger.write(WINKEL_MIN);
  delay(500);
}

int lesePotiGemittelt() {
  long summe = 0;
  for (byte i = 0; i < ANZAHL_MESSUNGEN; i++) {
    summe += analogRead(POTI_PIN);
    delay(2);
  }
  return summe / ANZAHL_MESSUNGEN;
}
```

```

void loop() {
  int rohwert = lesePotiGemittelt();
  int begrenzt = constrain(rohwert, ADC_LEER, ADC_VOLL);

  int prozent = map(begrenzt, ADC_LEER, ADC_VOLL, 0, 100);
  int winkel = map(begrenzt, ADC_LEER, ADC_VOLL, WINKEL_MIN, WINKEL_MAX);

  if (abs(winkel - letzterWinkel) >= TOTBAND) { // Nadel steht ruhig
    zeiger.write(winkel);
    letzterWinkel = winkel;
  }

  Serial.print(rohwert);
  Serial.print('\t');
  Serial.print(prozent);
  Serial.print('\t');
  Serial.println(winkel);

  delay(50);
}

```

6 – Typische Fehler und ihre Ursache

Beobachtung	Ursache und Abhilfe
LED lässt sich nicht mehr dimmen, seit der Servo im Programm ist	<code>Servo.h</code> belegt Timer 1 – damit ist <code>analogWrite()</code> auf Pin 9 und 10 blockiert. LED auf 3, 5, 6 oder 11 legen.
Nadel zittert dauernd um einen Wert	ADC-Rauschen wird direkt in Winkel umgesetzt. Mittelwert bilden und Totband einbauen (siehe Sketch).
Servo brummt am Anschlag und wird warm	Sollwinkel liegt außerhalb des mechanisch Erreichbaren. Bereich auf 5° bis 175° begrenzen, Hebel prüfen.
Arduino startet neu, sobald der Servo anfährt	Anlaufstrom bricht die 5 V ein. Eigene Versorgung, Massen verbinden.
Servo läuft endlos im Kreis	Kein Standardservo, sondern ein Modell mit Dauerdrehung (<i>continuous rotation</i>). Dort steuert <code>write()</code> die Drehrichtung, nicht den Winkel.
Anzeige springt in Stufen	Normal: 1024 ADC-Stufen auf 170° Anzeigeweg. Bei nur teilweise genutztem Potibereich entsprechend weniger – genau die Frage aus AB 7.

Wenn es genauer sein muss

Statt `write(winkel)` lässt sich mit `writeMicroseconds(1000...2000)` direkt die Pulsdauer setzen. Das umgeht die ganzzahlige Gradauflösung und ist bei einer Anzeigenadel spürbar feiner. Die mechanische Genauigkeit des Servos setzt dem allerdings schnell eine Grenze – ein SG90 hält seine Position nur auf etwa 1° genau.

Muster-Sketch zum Download:

https://hbraak.github.io/fachpraxis/arduino/beispiel_servo_zeigerinstrument.ino